

Module 5. Vulnerability Assessment

Vulnerability assessments scan networks for known security weaknesses: it recognizes, measures, and classifies security vulnerabilities in a computer system, network, and communication channel; and evaluates the target systems for vulnerabilities such as missing patches, unnecessary services, weak authentication, and weak encryption. Additionally, it assists security professionals in securing the network by determining security loopholes or vulnerabilities in the current security mechanism before attackers can exploit them.

The information gleaned from a vulnerability assessment helps you to identify weaknesses that could be exploited and predict the effectiveness of additional security measures in protecting information resources from attack.

Objective

The objective of this lab is to extract information about the target system that includes, but not limited to:

- Network vulnerabilities
- IP and Transmission Control Protocol/User Datagram Protocol (TCP/UDP) ports and services that are listening
- Application and services configuration errors/vulnerabilities
- The OS version running on computers or devices
- Applications installed on computers
- Accounts with weak passwords
- Files and folders with weak permissions
- Default services and applications that may have to be uninstalled
- Mistakes in the security configuration of common applications
- Computers exposed to known or publicly reported vulnerabilities

Vulnerability Classification

Vulnerability Type	Description	Examples
Misconfigurations/Weak Configurations	• Misconfigurations occur when systems, applications, or devices are not configured correctly, leaving them susceptible to exploitation • It allows attackers to break into a network and gain unauthorized access to systems	Network Misconfigurations • Insecure protocols, open ports and services, errors, and weak encryption
		Host Misconfigurations • Open permissions and unsecured root accounts
Application Flaws	• Application flaws are vulnerabilities in applications that are exploited by attackers • Flawed applications pose security threats such as data tampering and unauthorized access to configuration stores	• Buffer overflows, memory leaks, resource exhaustion, integer overflows, null pointer/object dereference, DLL injection, race conditions, improper input handling, improper error handling, and code signing weakness
Poor Patch Management	• Software vendors provide patches that prevent exploitations and reduce the probability of threats exploiting a specific vulnerability • Unpatched software can make an application, server, or device vulnerable to various attacks	• Unpatched servers, unpatched firmware, unpatched OS, and unpatched applications
Design Flaws	• Logical flaws in the functionality of the system are exploited by the attackers to bypass the detection mechanism and acquire access to a secure system	• Incorrect encryption and poor validation of data
Third-Party Risks	• Third-party services can have access to privileged systems and applications, through which financial information, customer and employee data, and processes in the enterprise's supply chain can be compromised	• Vendor management, supply-chain risks, outsourced code development, data storage, and cloud-based vs. on-premises risks

Misconfiguration : Occurs when systems, applications, or devices aren't configured properly

Insecure Protocols: Transmit information or data in plaintext

Types of Misconfigurations

1. Network
2. Host

Examples of Network Misconfigurations

1. Insecure Protocols
2. Open Ports and Services
3. Errors
4. Weak Encryption

Example of Host Misconfigurations

1. Open Permissions
2. Unsecured Root Accounts

Application Flaws: Vulnerabilities in Applications

Buffer Overflow: Occurs when an application receives more input than it expects

Memory Leak: Occurs when a programmer forgets to erase an assigned block of memory when it is no longer needed

Integer Overflow: Occurs when a function generates or attempts to store an integer larger than the maximum value for the allocated space

Null Pointer/Object Deference: Occurs when a value stored to represent a pointer isn't designated to an object

DLL Injection: An attack that injects a Dynamic Link Library (DLL) into memory and runs it. Attackers rewrite DLL, inserting malicious code

Race Conditions: Occurs when a program depends on the execution of processes in a sequence and the program is coerced to perform the execution of these processes simultaneously

Code Signing Weakness: Occurs when digitally signed software has a certificate that lacks validity, has expired, or originated from an unauthorized authority

Examples of Software Misconfigurations

1. Buffer Overflows
2. Memory Leaks
3. Resource Exhaustion
4. Integer Overflows
5. Null Pointer/Object Deference
6. DLL Injection
7. Race Conditions: Time of Check/Time of Use
8. Improper Error Handling
9. Code Signing Weakness

Examples of Poor Patch Management

1. Unpatched Servers
2. Unpatched OS
3. Unpatched Applications
4. Unpatched Firmware

Examples of Third Party Risks

1. Vendor Management: System Integration, Lack of Vendor Support
2. Supply Chain Risks
3. Outsourced Code Development
4. Data Storage
5. On-Cloud vs On-Premise

Types of Vulnerabilities

1. Misconfigurations
2. Application Flaws
3. Poor Patch Management
4. Design Flaws
5. Third Party Risks

6 Default Configurations

Using out-of-the-box settings that are widely known and easily exploitable.

7 OS Flaws

Vulnerabilities in the operating system that can be leveraged for privilege escalation or control.

8 Default Passwords

Pre-set credentials that attackers often try first during brute-force attempts.

9 Zero-Day Vulnerabilities

Newly discovered flaws with no official fix or patch available yet.

10 Legacy Platform Vulnerabilities

Security issues in outdated systems no longer supported by vendors.

11 System Sprawl/Undocumented Assets

Untracked or forgotten systems that escape regular monitoring and updates.

12 Improper Certificate and Key Management

Weak or exposed cryptographic keys and certificates compromising secure communications.

- *CVSS*: Measures severity with base, temporal, and environmental scores
- *CVE*: Unique identifiers for vulnerabilities
- *NVD*: Enriched CVE data, includes CVSS scores
- *CWE*: Categorizes root causes (e.g., buffer overflow)

Common Vulnerability Scoring System (CVSS)

- CVSS helps capture the principal characteristics of a vulnerability and produces a numerical score to reflect its severity

CVSS Metric Groups

Base metric: Represents the intrinsic characteristics of a vulnerability that are constant over time and across user environments.

o Exploitability metrics: Reflects the ease and technical means by which the vulnerability can be exploited.

- Threat metric: Reflects the characteristics of a vulnerability related to threat that may change over time but not necessarily across user environments.
- Environmental metric: Represents the characteristics of a vulnerability that are relevant and unique to a particular consumer's environment.
- Supplemental metric: Includes metrics that provide context as well as describe and measure additional extrinsic attributes of a vulnerability.

CVSS Exploitability Metrics

1. Attack Vector
2. Attack Complexity
3. Attack Requirements
4. Privileges Required
5. User Interaction

CVSS Impact Metrics

- Vulnerable System Confidentiality, Integrity, Availability
- Subsequent System Confidentiality, Integrity, Availability

CVSS Supplemental Metrics

1. Automatable
2. Recovery
3. Safety
4. Value Density
5. Vulnerability Response Effort
6. Provider Urgency

CVSS Base Score Severity

- 0: None
- 0.1-3.9: Low
- 4.0-6.9: Medium
- 7.0-8.9: High
- 9.0-10.0: Critical

CVE

Provides an identifier for each vulnerability or exposure and a standardized description. It is a way to provide interoperability, a basis for evaluation, and a method for disparate databases to speak the same language.

National Vulnerability Database (NVD)

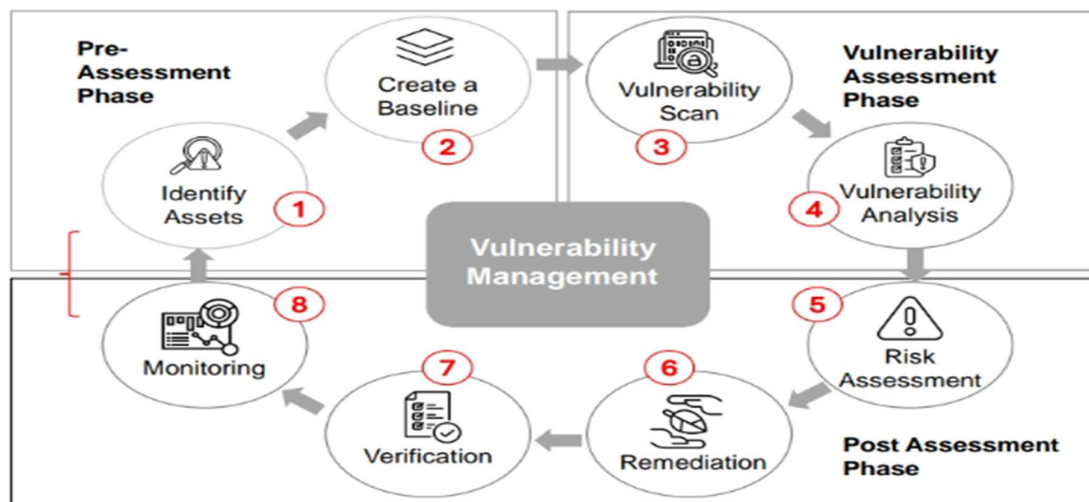
- NVD is the U.S. government repository of standards-based vulnerability management data
- NVD performs an analysis on CVEs that have been published to the CVE Dictionary

<https://nvd.nist.gov>

Common Weakness Enumeration (CWE)

- CWE is a category system for software vulnerabilities and weaknesses
- CWE's over 600 categories of weaknesses provide an effective baseline for the community's identification, mitigation, and prevention efforts

<https://cwe.mitre.org>



Pre-Assessment Phase Steps

1. Identify and Understand Business Processes
2. Identify the applications, data, and services that support the business processes and perform code reviews
3. Identify the approved software, drivers, and basic configuration of each system
4. Create an inventory of all assets, and prioritize or rank the critical assets
5. Understand the network architecture and map the network infrastructure
6. Identify the controls already in place
7. Understand policy implementation and practice standard compliance with business processes
8. Define the scope of the assessment
9. Create information protection procedures to support effective planning, scheduling, coordination, and logistics

Steps in the Vulnerability Assessment Phase

1. Examine and evaluate the physical security
2. Check for misconfigurations and human errors
3. Run vulnerability scans using tools
4. Select the type of scan based on the organization or compliance requirements
5. Identify and prioritize vulnerabilities
6. Identify false positives and false negatives

7. Apply the business and technology context to scanner results
8. Perform OSINT information gathering to validate the vulnerabilities
9. Create a vulnerability scan report

Post-Assessment Phase Steps

1. Risk Assessment
2. Remediation
3. Verification
4. Monitoring

Risk Assessment Tasks

- Creating a priority list for assessment recommendations based on the impact analysis
- Developing an action plan to implement the proposed remediation
- Capturing lessons learned to improve the complete process in the future
- Conducting training for employees

Remediation Tasks

- Prioritize remediation based on the risk ranking
- Develop an action plan to implement the recommendation or remediation
- Perform a root-cause analysis o Apply patches and fixes
- Capture lessons learned o Conduct awareness training
- Perform exception handling and risk acceptance for the vulnerabilities that cannot be remediated

Verification Tasks

- Rescanning Systems
- Performing Dynamic Analysis
- Reviewing the Attack Surface

Monitoring Tasks

- Periodic vulnerability scan and assessment
- Timely remediation of identified vulnerabilities
- Monitoring intrusion detection and intrusion prevention logs
- Implementing policies, procedures, and controls

Vulnerability Scanning and Analysis

- 1 Vulnerability scanning involves analyzing protocols, services, and configurations to discover vulnerabilities and design flaws that may expose an operating system and its applications to exploitation, attack, or misuse
- evaluating, and prioritizing security weaknesses in systems, networks, applications, or protocols
- 3 Vulnerabilities are classified based on severity level (low, medium, or high) and exploit range (local or remote)
- 4 The goal of this analysis is to understand the nature of these vulnerabilities, assess their potential impact, and develop strategies to mitigate or eliminate t

Types of Vulnerability Scanning

1 External Scanning Scans the network from a hacker's perspective to discover exploits and vulnerabilities that are accessible to the outside world	4 Network-based Scanning Determines possible network security attacks that may occur on the organization's systems	7 Non-Credentialed Scanning A security testing method that assesses systems, networks, and applications without using valid credentials to log into the target system
2 Internal Scanning Scans the internal infrastructure to discover exploits and vulnerabilities	5 Application Scanning Tests and analyzes all elements of the web infrastructure for any misconfiguration, outdated content, or known vulnerabilities	8 Manual Scanning Manually identifying, evaluating, and validating security vulnerabilities in systems, networks, and applications
3 Host-based Scanning Conducts a configuration-level check to identify system configurations, user directories, file systems, registry settings, etc., to evaluate possibility of compromise	6 Credentialed Scanning Scanner logs into the target system using valid credentials to perform a more thorough and comprehensive scan	9 Automated Scanning Uses automated software tools such as Nessus, Qualys, and GFI LanGuard to systematically identify, evaluate, and report security vulnerabilities

Approaches to Vulnerability Assessment

- 1. Product-Based:** Uses commercial or open-source tools installed within the organization to identify vulnerabilities.
- 2. Service-Based:** Involves external experts or services to conduct vulnerability assessments remotely or onsite.
- 3. Tree-Based:** Auditor follows a predefined decision tree or strategy per component/system to conduct the assessment methodically.
- 4. Inference-Based:** Scanning starts by building an inventory of the protocols found on the machine. After finding a protocol, the scanning process starts to detect which ports are attached to services, such as an email server, web server, or database server. After finding services, it selects vulnerabilities on each machine and starts to execute only those relevant tests.

Vulnerability Solution Steps

Locating nodes: The first step in vulnerability scanning is to locate live hosts in the target network using various scanning techniques.

- Performing service and OS discovery on them: After detecting the live hosts in the target network, the next step is to enumerate the open ports and services along with the operating system on the target systems.

Testing those open services and the operating system running on the target nodes, they are tested for known vulnerabilities.

Types of Vulnerability Assessment Tools

There are six types of vulnerability assessment tools:

- 1. Depth Assessment Tool:** Used to discover and identify previously unknown vulnerabilities in a system
- 2. Host-Based Tools** – Analyze individual systems for OS-level and configuration vulnerabilities.
- 3. Application-Layer Tools** – Detect flaws in web and software applications like XSS or SQL injection.
- 4. Scope Assessment Tools** – Evaluate a wide range of systems across the network for broader coverage.

5. Active and Passive Tools – Actively probe systems or passively monitor traffic to identify vulnerabilities.
6. Location and Data Examination Tools
 - Network-Based Tool: Interact only with the real machine where they reside and give the report to the same machine after scanning
 - Agent-Based: Reside on a single machine but can scan several machines on the same network
 - Cluster Scanners: Similar to proxy scanners, but they can simultaneously perform two or more scans on different machines in the network
 - Proxy Scanners: Network-based scanners that can scan networks from any machine on the network

Tool	Type	Notes
OpenVAS	Scanner / Framework	Comprehensive NVT scanning; supports authenticated/unauthenticated scans.
Nessus	Scanner	Performs OS, network, config, and compliance checks; runs on port 8834.
GFI LanGuard	Vulnerability / Patch Management	Detects misconfigs & missing patches on network devices.
Nikto	Web Scanner	Assesses web servers for misconfigs, outdated software, CGI flaws.
Qualys	Cloud Scanner (SaaS)	Provides scalable, cloud-based asset & vulnerability management.
Nmap + NSE Scripts	Port/Service/Discovery	Discovers open ports/services; NSE scripts enable vuln detection.
ShellGPT / AI Tools	AI-Assisted Scanner	Automates tool use via natural language; generates commands/results.

Tool	Type	Notes
SmartScanner	AI Website Scanner	Uses ML/AI to detect and mitigate web vulnerabilities.
Equixly	API Security SaaS	Integrates with dev workflows for AI-based API security testing.
CodeDefender	Web & Code Analysis	Focuses on securing web code with vulnerability insights.
DryRun Security	Dev Security Automation	Automates early-stage app vulnerability detection.
Hackules	Web Security Scanner	AI-powered scanner for websites and APIs.
Corgea	SaaS Security Tool	Offers AI-based website vulnerability assessments.
Pentest Copilot	AI Pentest Assistant	Assists bug bounty hunters with automated pentesting tasks.

Vulnerability Assessment Reports

A vulnerability assessment report is a **comprehensive document** that details the findings of a vulnerability assessment

Executive Summary ▪ Assessment scope and objectives ▪ Testing narrative ▪ Findings summary ▪ Remediation summary ▪ Component compliance summary	Risk Assessment ▪ Classification of vulnerabilities based on the risk level ▪ Potential vulnerabilities that can compromise the system or application ▪ Critical hosts with severe vulnerabilities	Appendices and Supporting Information ▪ Additional information that supports the report's findings such as detailed logs, configuration files, or references to external resources
Assessment Overview ▪ Assessment methodology ▪ Scan information ▪ Target information ▪ Tools involved	Recommendations ▪ Prioritization of remediation based on the risk ranking ▪ Action plan to implement the recommendations for each identified vulnerability ▪ Root-cause analysis ▪ Application of patches/fixes ▪ Lessons learned ▪ Awareness training ▪ Implementation of periodic vulnerability assessment ▪ Implementation of policies, procedures, and controls	Conclusion ▪ Summary of key findings and recommendations, reinforcing the importance of addressing the identified vulnerabilities
Findings ▪ Scanned hosts ▪ Affected assets ▪ Types of vulnerabilities identified ▪ Detailed information on identified vulnerabilities ▪ Notes describing additional details of scan results		Follow-Up Actions and Timeline ▪ Timeline for re-assessment or follow-up actions to ensure vulnerabilities are addressed and to monitor the effectiveness of the remediation efforts
		Glossary of Terms ▪ Definitions for technical terms used in the report